

Glossary — the field's vocabulary, and which of it we may honestly claim

Version 3.71.0 · Last updated 2026-08-07

Will, 2026-08-06: "INCORPORATE THIS FANCY TERMINOLOGY IN OUR DOCUMENTS SO WE KNOW TO USE IT IN OUR PAPER."

Every term here is standard in the literature. Each entry says what it means, where **we** already have it, and — where it matters — **what we may not claim**. A paper that misuses one of these is a paper a reviewer stops trusting, and several of them are easy to overstate in exactly the direction that flatters us.

1. The game, formally

Zero-sum — one player's gain is the other's loss. VGC is, barring the turn-limit tie.

Imperfect information — you cannot see the full state. In VGC: their exact spreads, which four of six they brought, the fourth move on a set. *Distinct from stochastic*, which is the dice. VGC is both, and conflating them is the most common error in this literature.

Simultaneous move — both players commit before either resolves. This is why minimax is not sound here and why the solution concept is a mixture. Chess and Go are sequential; poker is sequential with hidden information; **VGC is simultaneous AND hidden AND stochastic**, which is why no off-the-shelf recipe ports cleanly.

Team configuration space — VGC-Bench estimates $\sim 10^{139}$, stated as larger than Chess, Go, Poker, StarCraft or Dota. Citable, and the best one-line answer to *why is this hard*.

2. Solution concepts — the part we must not overstate

Nash equilibrium — a strategy profile where neither player gains by deviating alone. In a two-player zero-sum game it is the min-max point: it maximises your **worst case**.

Mixed strategy — a probability distribution over actions rather than one action. In imperfect information the equilibrium is *generally* mixed, which is the formal reason "always click the best move" is wrong.

GTO (game-theory optimal) — poker's word for playing the equilibrium. It does **not** mean "wins the most". It means "cannot be beaten by anyone", and it deliberately gives up EV against weak opponents in exchange.

Exploitative play — the best response to your *model* of this opponent. Maximises EV if the model is right; collapses when it is wrong, and is itself exploitable. **Modern poker agents play near-GTO and deviate to exploit only on strong evidence.**

*The distinction ADR-003 turns on. VGC-Bench optimised for winning against a training distribution — exploitative in effect — and measured **~100% exploitability**. Our thesis is that a resolving agent is harder to exploit than a compiled one. That is a claim about the shape of the agent, not about its win rate.*

Exploitability — how much a **best response** trained against you can beat you by. It is **intrinsic**: measured against an exploiter trained on *you*, in *your* format. This is why we can compare numbers with VGC-Bench even though their checkpoints are Reg M-A and ours is Reg M-B and the two agents can never meet.

ϵ -exploitability / approximate equilibrium — the honest framing for any real system. **We cannot compute the equilibrium of a game this size and must never say we have.** Heads-up *limit* poker was essentially solved (Cepheus, 2015); heads-up *no-limit* never was — it was approximated well enough to beat professionals. VGC doubles is larger.

WHAT WE MAY CLAIM: *exploitability reduced from X to Y, measured the way they measured theirs.* **WHAT WE MAY NOT CLAIM:** *"unexploitable", "solved", "equilibrium", or "perfect knowledge of all plausible scenarios". We will have a sample of plausible scenarios, and the sample's quality is the engineering problem.* **Equilibrium is a direction, not a destination.**

3. Learning methods — what the other projects did

Behaviour cloning (BC) — supervised imitation: for every observed position, train the model to output the click the human made. Caps at human-average **by construction**, because that is its target. → **Ours is MAG**, fitted to predict human clicks. We have had BC all along under another name.

PPO (Proximal Policy Optimization) — the standard policy-gradient RL algorithm. Reinforce winning actions, discourage losing ones, with a clipped update so a single batch cannot destroy the policy. → **Ours is MACHAMP**, the self-play improvement loop.

Self-play (SP) — train against a copy of your current self. Beat Go. In imperfect-information games it risks co-evolving into a bubble: excellent against yourself, exploitable by anything else.

Fictitious play (FP) — train against a *uniform draw from your own history*, so you must stay good against every past version rather than only the present one.

Double oracle (DO) — same pool, but sample opponents from the **Nash mixture over the pool**, solved as a linear program. The game-theoretic member of the family. → **Ours is SLOWKING**, which already solves preview this way.

PSRO (Policy-Space Response Oracle) — the unifying framework SP/FP/DO are special cases of. Worth naming once in the paper; VGC-Bench cites it as future work.

BCSP / BCFP / BCDO — VGC-Bench's notation: behaviour cloning **then** PPO fine-tuning under each of the three. **BCSP** was their strongest. Their measured spread is the useful part: **BC alone wins 0.130 against BCSP and SP alone wins 0.246** — the two stages together are worth far more than either half.

Offline RL — learn from a fixed dataset with no environment interaction. → Metamon (RLC 2025), top 10% in singles **with no search at all**, which is live counter-evidence to our thesis and must be cited as such.

4. Search

Minimax / expectiminimax — exhaustive adversarial search, the latter with chance nodes. **Not sound under simultaneous moves**, which is why Foul Play left it.

MCTS (Monte Carlo Tree Search) — expand promising branches, sample the rest. Foul Play uses **root-parallelised** MCTS *"guided by a custom evaluation function rather than rollouts"* — which is ROADMAP #24, in production, at top-100.

Rollout / playout — simulating to the end and scoring the result. **Ours does this and it is the thing #24 replaces:** MILTANK's leaf plays 200 games of uniform-random clicks and loses to a coin.

Evaluation function / value function — score a position without playing it out. → **PORYGON2**.

Depth-limited solving / continual re-solving — poker's answer (DeepStack, ReBeL): do not search to the end; search a few plies and use a learned value at the frontier, re-solving every decision. **This is the shape ADR-003 commits us to.**

Public belief state — the state of the game *plus* both players' beliefs, treated as the object you solve over. ReBeL's central idea and the formal home for what GARY and XATU approximate.

Root parallelisation — run independent searches from the root on many cores and combine. Scales **sublinearly**: 16× cores does not buy 16× depth. Relevant because we use one core of sixteen.

CFR (counterfactual regret minimization) — the algorithm that solved poker. Minimises regret per information set and converges to equilibrium. `docs/POKER-TO-POKEMON.md` works through the correspondence.

Abstraction — shrink the game (bucket hands, discretize bets) so a solver fits, then map back. The reason poker was tractable at all, and an open question for us: **what is VGC's abstraction?**

5. Testing — the vocabulary of our actual contribution

Differential testing (McKeeman 1998) — feed the same input to two implementations of one spec and compare. **The oracle is free.** Strictly stronger than fuzzing, which sees only crashes.

Test oracle — the thing that decides whether an output is correct. Ours is the official Showdown engine (ADR-002).

Test harness — the rig a system is strapped into so it can be driven and recorded. Borrowed from hardware: a *wiring harness* connects a component to the measuring equipment.

Swarm testing (Groce et al., ISSTA 2012) — many configurations, each **omitting** features. Found 42% more distinct compiler crashes than a hand-tuned default. **Feature omission** is the mechanism: some features suppress the behaviour you are trying to reach. **Ours is Protect**, on 99.3% of teams.

Directed swarm testing (Groce et al., ISSTA 2016) — use a swarm's own statistics to bias generation toward rarely-covered targets. The feedback loop in `docs/GAME-DIFFERENTIAL-DESIGN.md` §3.2.

Delta debugging / `ddmin` (Zeller & Hildebrandt) — shrink a failing input to a **1-minimal** one, where removing any remaining element makes the failure vanish. Turns a divergence into a WIRE.

Metamorphic testing — when no oracle exists, assert *relations* between outputs instead. Named because reviewers will ask why we did not need it: **we have a real oracle.**

Mutation testing — deliberately break the system and check a test notices. → our red-demonstration rule: **no probe counts until it has been shown failing on a broken engine.** 122 demonstrations, 0 failed.

Ratchet — a monotone bound: a count that may shrink and may never grow. Not standard literature vocabulary; ours, and worth defining explicitly in the paper because it is load-bearing.

Steering input — a file that decides WHICH tests get run, as opposed to what they check. Ours is `data/mechanics-census.json`: the differential's driver prefers the action reaching the least-exercised census row, so regenerating the census changes the sample. A steering input is inside the photograph even though no measuring code ever opens it, which is why `engine/steering.js` declares and digests it and `engine/arms_comparable.js` refuses a pair whose steering differs. Ours; the general form of the idea is *coverage-guided generation*, but the failure mode — an uncontrolled sample from a file nobody listed — is what earned it a name here.

Release ladder — running one instrument over EVERY frozen release of a change series under a single pinned input, so that all arms are mutually comparable rather than only adjacent. `engine/wire_ladder.js`. It buys two things a chain of pairwise before/after cannot: an effect landing between two named changes is attributed to itself rather than to whichever change came next, and the whole series is read against one baseline instead of against a moving one.

6. Evaluation and honesty

Proper scoring rule — a score that is optimised by reporting your true belief. **Log-loss** and **Brier** are proper; accuracy is not. Every probability we publish carries one.

Calibration / ECE — when you say 70%, does it happen 70% of the time? Expected Calibration Error is the summary. A model can discriminate well and be badly calibrated, and vice versa.

Baseline — what the result must beat to mean anything. Ours are a coin, material heuristics, and strong-player play. **A result without a baseline is not a result.**

Held-out / split-half noise floor — the smallest difference distinguishable from noise. Ours is **0.43 points**; anything under it is not a finding.

Common random numbers / paired comparison — give both arms identical seeds so the difference is the treatment rather than luck. Standard variance reduction, and how our head-to-heads are run.

Elo / Glicko — ladder rating. Sarantinos' caveat is load-bearing: comparing human and AI ratings is invalid **unless they played each other**. Laddering with ALAKAZAM is that, so our ladder result is a legitimate comparison and should say why.

Anecdotal evaluation — VGC-Bench's own word for their human tests (5 games each against three players). Worth quoting when we describe their result, because it is the honest frame and it is theirs.

7. Terms we use that the field does not

Defined here so the paper can introduce them once rather than leaking jargon.

ours	means
WIRE	a numbered engine defect found and fixed, with a red demonstration attached
the census	the per-mechanic table of probed / live / armed / directCall
armed	a probe that has been shown RED on a deliberately broken engine
the differential	the step-level protocol comparison against the official engine
frozen release	a byte-copy of 23 engine files, so a measurement reads a photograph and not a moving tree
the photograph rule	nothing in frame may move during a measurement, including files the measurer never opens
RAW-STORE-OK	a declared, reasoned exception to a rule, rather than a silent one

Sources

- McKeeman, *Differential Testing for Software* — <https://www.cs.tufts.edu/comp/150FP/archive/bill-mckeeman/DifferentailTesting.pdf>
- Yang, Chen, Eide, Regehr, *Finding and Understanding Bugs in C Compilers* (PLDI 2011) — <https://users.cs.utah.edu/~regehr/papers/pldi11-preprint.pdf>
- Groce et al., *Swarm Testing* (ISSTA 2012) — <https://agroce.github.io/issta12.pdf>
- Groce et al., *Directed Swarm Testing* (ISSTA 2016) — <https://agroce.github.io/issta16.pdf>

- Zeller & Hildebrandt, *Simplifying and Isolating Failure-Inducing Input* — <https://www.computer.org/csdl/journal/ts/2025/03/10859156/23X97jMgYjm>
- Angliss, Cui, Hu, Rahman, Stone, *VGC-Bench* (AAMAS 2026) — <https://arxiv.org/abs/2506.10326>
- Grigsby, Xie, Sasek, Zheng, Zhu, *Human-Level Competitive Pokémon via Offline RL* (RLC 2025) — <https://arxiv.org/abs/2504.04395>
- Hu, Huang et al., *PokéLLMon* — <https://arxiv.org/abs/2402.01118>
- Sarantinos, *Teamwork under extreme uncertainty* — <https://arxiv.org/pdf/2212.13338>
- Brown & Sandholm, *Libratus*; Moravčík et al., *DeepStack*; Brown et al., *ReBeL* — see `docs/POKER-TO-POKEMON.md`